Building a Graph Neural Network based Movie Recommendation System

Indiana University -Bloomington

Abstract

Since the early part of this century, systems that can automatically recommend items to users have become extremely important. One of the most common and effective ways to create these recommendation systems in recent years is with a graph structure that uses machine learning to model a user's preferences for different items. There are many different learning algorithms for this task, but in this paper we focus on three of them: Shallow Embedding, Neural Graph Collaborative Filtering, and Light Graph Convolutional Networks. In this paper, we provide a survey of these three models on one of the most common movie recommendation dataset. First, we provide our findings from these models and compare the results to each other. We also provide a demo system that can recommend the top *n* movies for any given user in the dataset.

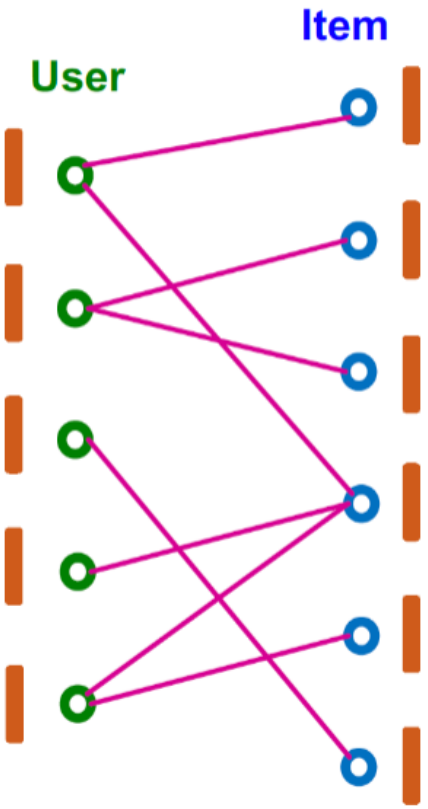


Figure 1: Example of bipartite graph between users and items for a recommendation system [8]

1 Introduction

Since the early days of the modern internet, many companies have attempted to build algorithms that can automatically recommend new items to users based only on interaction data with other items. Recommendation system uses vary widely depending on the applications. Online sellers such as Amazon or Walmart use these systems to suggest new products for a shopper to buy based on products bought from their store previously. Social networks try to recommend new "friends" to a user that are related to people they are already connected to. Finally, movie streaming platforms like Netflix or Hulu recommend movies that are similar to other shows that a user has liked in the past. This is a very important task for many different companies as successfully predicting a user's interests can greatly increase the success of that company. However, the difficulty comes in the fact that it is very difficult to recommend an item purely based off of gathered data.

There are many different algorithms and structure that have been used to make these recommendation systems. The most common way today is to utilize a graph structure to represent user-item interactions and then use advanced machine learning techniques to learn from that data and predict new connections. The graph is set up as a bipartite graph with users on one side and items on the other side. An edge between a user and an item represents an interaction between that user and the corresponding item. For example, if

User A watches Game of Thrones, there will be an edge between User A and Game of Thrones in the graph. An illustration of this type of graph is shown above in figure 1.

By using this type of data structure, the questions of "How can we make efficient item recommendations?" becomes a questions of link prediction on the bipartite graph. Because of this, models are able to leverage the efficiency of graph-based algorithms to make recommendations [1].

Our procedure in this paper is:

1. Implement three different graph recommendation systems and analyze their results on a movie recommendation dataset.
2. Build a simple movie recommendation system that can recommend movies for any user in the data set based off of their previously watched movies.

2 Method

In this section, we will start by explaining the loss function that is used for our models. We will then talk about each of our three models: Shallow Embedding, NGCF, and LightGCN. We will then discuss an additional model that we attempted to implement but were not able to finish. Finally, we will talk about how the embeddings generated in each of the models can be used to make recommendations.

2.1 BPR Loss

The loss function that we use for all of our models is Bayesian Personal Ranking (BPR) loss. The formula for it is:

$$BPR(u) = \frac{1}{|E| \cdot |E_{neg}|} \sum_{(u,v_{pos}) \in E(u)} \sum_{(u,v_{neg}) \in E_{neg}(u)} -\log\left(\sigma(f_\theta(u, v_{pos}) - f_\theta(u, v_{neg}\right)$$

where $E(u)$ are the positive edges of user u, and $E_{neg}(u)$ are the negative edges of user u. The total BPR loss can the be calculated by $Loss = \frac{1}{|U|} \sum_{u^* \in U} BPR(u^*)$ to get the average for the whole network [8].

The reason for using this loss function is that it aligns much better with the recall metric, which is the primary measurement of success for recommendation systems. It essentially makes it so that the model learns to score positive edges higher than negative edges [8]. This makes it so that the model focus on learning patterns between users and items that were interacted with, without spending effort looking at non-existent relations.

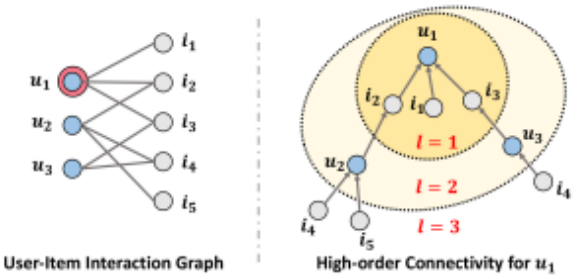


Figure 2: Visual example of how a user u can be connected to multiple "hops" of neighbors through the higher order connectivity Of graph convolutional networks [2]

2.2 Shallow Embedding

The simplest method that we use in this paper is shallow node embedding. For each node we calculate several "random walks" across the network. A raodnom walk is a fixed-size series of connected nodes starting from a given node A [7]. The purpose of these walks is to find how often other nodes are included in the random walks and using that to model graph structure.

Once we have these random walks for the nodes in the graph, we use the Word2Vec[5] model to transform this information into node-specific embeddings. This produces a shallow encoding for all of the nodes in the network.

2.3 NGCF

Neural Graph Collaborative Filtering (NGCF)[2] uses a Graph Neural Network (GNN) to produce node embeddings. The main idea motivation behind this model is that the random walk techniques described above are not able to explicitly capture graph structure, which causes some information about the data to be lost. The NGCF architecture uses GNNs because they are capable of capturing embedding that are explicitly aware of the high-order graph structure [8]. An example of this is shown in figure 2.

The mathematics behind the model are as follows:

$$\mathbf{e}_u^{(k+1)} = \sigma\left(\mathbf{W}_1\mathbf{e}_u^{(k)} + \sum_{i\in\mathcal{N}_u}\frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}}(\mathbf{W}_1\mathbf{e}_i^{(k)} + \mathbf{W}_2(\mathbf{e}_i^{(k)}\odot\mathbf{e}_u^{(k)}))\right),$$
$$\mathbf{e}_i^{(k+1)} = \sigma\left(\mathbf{W}_1\mathbf{e}_i^{(k)} + \sum_{u\in\mathcal{N}_i}\frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}}(\mathbf{W}_1\mathbf{e}_u^{(k)} + \mathbf{W}_2(\mathbf{e}_u^{(k)}\odot\mathbf{e}_i^{(k)}))\right),$$

Here, $e_i^{(k+1)}$ is the embedding of item *i*, and $e_u^{(k+1)}$ is the embedding of uses *u.* $\boldsymbol{\sigma}$ is the activation function. W_1 and W_2 are the trainable weight matrices for the network. N_u and N_i are the set of items that are interacted with by user *u,* and the set of users that interact with item *i.* respectively [3].

Thes formulas show that the model uses message passing to propagate information from a given node to its neighboring nodes using the GNN. Information can be passed to further nodes by using more layers in the GNN [2]. This message pass structure is what allows the model to learn higher order structure explicitly since it is directly learning information from its neighbors in the network.

2.4 LightGCN

The LightGCN model [3] is built with the same general idea of NGCF, except with additional assumptions. The authors of that paper noticed that shallow learnable embeddings are already very good at capturing information. As a result, they simplified the NGCF formula by removing it's learnable weights and got rid of nonlinearity in the model [8]. The formula for the model is:

$$\mathbf{e}_u^{(k+1)} = \sum_{i\in\mathcal{N}_u}\frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}}\mathbf{e}_i^{(k)},$$
$$\mathbf{e}_i^{(k+1)} = \sum_{u\in\mathcal{N}_i}\frac{1}{\sqrt{|\mathcal{N}_i|}\sqrt{|\mathcal{N}_u|}}\mathbf{e}_u^{(k)}.$$

where $e_i^{(k+1)}$, $e_u^{(k+1)}$, N_u, and N_i are the same as in the NGCF model [3]. As can be seen by comparing this formula to the NGCF formula, LightGCN kept the same general steps but greatly simplified the calculations.

2.5 GC-MC

Graph Convolutional Matrix Completion model [4] models the recommendation system problem differently. It involves an encoder decoder architecture where the encoder first tries to learn an embedding using different weight matrices for edges of different ratings and aggregates them:

$$\mu_{j\to i,r} = \frac{1}{c_{ij}}W_r x_j\,.$$

where (mu-j-ir) is the message from item j to user i for rating r, (Cij) is the normalization constant, (Wr) is the weight matrix and (xj) is the item embedding. The message is further transformed using another linear transform and an activation function.

The decoder simply predicts a probability distribution over all the ratings which is an Nu X Nv X r matrix, where Nu is the number of users and Nv is the number of items. The final predictions are given by:

$$\check{M}_{ij} = g(u_i, v_j) = \mathbb{E}_{p(\check{M}_{ij}=r)}[r] = \sum_{r\in R} r\; p(\check{M}_{ij}=r)\,.$$

which is the expedited value of the rating for the edge from i to j [4].

We were unfortunately not able to fully implement this model as we were too ambitious with our ideas and ran out of time before the project was due. However, we plan on implementing this in the future and comparing the results to our current models. Due to the complexity of it, we expect GC-MC to perform better than LightGCN.

2.6 Predictions

Once we have the embedding for a specific model, we then use them to generate recommendations. For a given user *u*, the score function for a given item *i,* the score function is simply the dot product of their shallow embeddings.

We calculate the score of a user with each of the items which is not already connected to a user. This is done so that we do not recommend an item that the user has already seen. The higher the score for an item, the more likely it is that a user would interact with it. Therefore, if we wanted to recommend the ten items that a given user would be most likely to interact with, we would take the ten items where the score between user and item is highest.

3 Experimental Setup

In this section, we discuss the dataset that we chose to use for our testing, as well as the details of the experiments that we ran for all of our models.

3.1 DataSet Description

The dataset that we use for this paper is the MovieLens 100k dataset [6]. This dataset collected from MovieLens, which is a movie recommendation service. It has 610 different users, 9,742 movies, and 100,836 connections between them. Each connection between two nodes is a rating from the user about a given movie. The ratings are from 1-star to 5-stars [6].

3.2 Experimental Settings

To set up our experiments, we first had to preprocess the data. For the purposes of this paper, we only kept edges that had a rating of at least 3-stars. That way, we would not be recommending movies that a user doesn't like.

We then split the connection data into training, testing, and validation sets. For each of the splits, we keep all of the same users and items. 80% of the edges are used for training, 10% are used for testing, and 10% are used for training. The reason that we only split the edges and not the nodes is because we want to treat this as a link prediction problem. If there were users not present in training for these models, the algorithm would simply predict the most popular movies for a user to watch since there was no data learned about them.

For each algorithm, we trained the model on the training data subset to learn relations between users and movies. After each epoch of training, the model was tested on the validation set of connections to ensure that it was not overfitting. Finally, after training completed, the model was tested on the testing subset and the final recall and precision were reported.

For model testing, we chose a k value of 5. This means that we recommend the top five movies that a user has not already interacted with. The recall is calculated by dividing the number of movies that were correctly guessed by the total number of positive edges in the testing set. Precision measures what percent of the predicted movies are actually correct. All experiment results are listed in the following section.

All models were run using Python3. We also used the most recent version of Pytorch and Pytorch-Geometric to build our models. The hardware system that our models were run on were our own personal Windows 10 PCs.

4 Results

For these experiments, we vary each hyperparameter to see the effects that each value has on the overall model. The hyperparameters are set to default values and only one of them is modified at a time to illustrate the changes.

For shallow embedding, the default values are 20 walks, a walk length of 20, and an embedding size of 10.

For NGCF, we use 64 hidden dimensions, 2 hidden layers, a learning rate of 0.001, and a batch size of 128.

For LightGCN, the default hyperparameters are 64 hidden dimensions, 2 hidden layers, a learning rate of 0.001, and a batch size of 128.

All of these hyperparameters were chosen as the default because they are what we found to perform the best early on in the testing process.

When working with the LightGCN we found that it was overfitting very easily. It was predicting the most frequently occurring movies for every user. While the precision and recall were high, the system was clearly not working well.

We came up with two strategies to tackle the same. We used a sampling strategy such that the less frequent the edge the more frequently it was sampled. While this gave us varied results it hurt the precision and recall tremendously (by a factor of 1/10).

We then used L2 regularization on the BPR Loss and also varied the alpha parameter (which is the weight multiplied to the embedding matrix at each layer when aggregating) which gave us sufficiently diverse results.

4.1 Shallow Embedding

For the shallow embedding, we experimented with three different hyper parameters. We found that there some difference when using different number of

Epochs	Hidden_dims	Num_layers	Batch_size	LRate	Lambda_reg	Alpha	Avg Precision	Avg Recall
200	128	3	256	0.01	0.05	0.33	0.16079	0.08689
200	128	3	256	0.01	0.033	0.33	0.19638	0.08948
200	128	3	256	0.01	0.025	0.33	0.20593	0.10286
200	128	3	256	0.01	0.02	0.33	0.20890	0.10391
200	128	3	256	0.01	0.05	0.5	0.20890	0.10391

Table 8: Hyperparameter studies on the LightGNC model.

random walks, walk lengths, and embedding sizes. However, none of them seem to perform very will since this algorithm does not explicitly capture the graph structure so learning is limited.

The best results that we were able to achieve with this model is a recall of 0.0371 and a precision of 0.533 when we use 20 walks of length 20 weight an embedding size of 84. Table 1, 2, and 3 show the results of the rest of the hyperparamter study.

Table 1: Hyperparameter study on number of walks

Num Walks	**10**	**20**	**50**	**75**	**100**
Recall	**0.302**	**0.0177**	**0.0156**	**0.0103**	**0.0114**
Precision	**0.0434**	**0.0254**	**0.0221**	**0.0148**	**0.0164**

Table 2: Hyperparameter study on walk length.

Walk Length	**10**	**20**	**50**	**75**	**100**
Recall	**0.0173**	**0.0158**	**0.0195**	**0.0149**	**0.0180**
Precision	**0.0246**	**0.0221**	**0.0279**	**0.0213**	**0.0254**

Table 3: Hyperparameter study on embedding size.

Emb. Size	**10**	**32**	**64**	**84**	**128**
Recall	**0.0137**	**0.0333**	**0.0313**	**0.0371**	**0.0318**
Precision	**0.0197**	**0.0475**	**0.0443**	**0.0533**	**0.0451**

4.2 NGCF

Table 4: Hyperparameter study on batch size.

Batch Size	**64**	**256**	**512**	**1024**	**2056**
Recall	**0.0787**	**0.0802**	**0.0804**	**0.0817**	**0.0808**
Precision	**0.0843**	**0.0866**	**0.0868**	**0.0883**	**0.0873**

Table 5: Hyperparameter study on learning rate.

LR	**0.0001**	**0.0005**	**0.001**	**0.005**	**0.1**
Recall	**0.0403**	**0.0641**	**0.0866**	**0.0841**	**0.0297**
Precision	**0.0427**	**0.0693**	**0.0929**	**0.0900**	**0.0315**

Table 6: Hyperparameter study on hidden dimension size.

Hidden Size	**32**	**64**	**128**	**256**	**512**
Recall	**0.0508**	**0.0805**	**0.0935**	**0.0938**	**0.0896**
Precision	**0.0544**	**0.0863**	**0.1003**	**0.1004**	**0.0974**

Table 7: Hyperparameter study on number of layers.

Num Layers	**4**	**6**	**8**	**10**	**12**
Recall	**0.0841**	**0.0091**	**0.0065**	**0.0067**	**0.0065**
Precision	**0.0920**	**0.0091**	**0.0069**	**0.0072**	**0.0070**

For NGCF, we were able to achieve much better results than shallow embeddings. Our model peaks at 0.0938 recall and 0.1004 precision when using the

default parameters and a hidden size of 256. The rest of the results are listed in the follow four tables. The validation curves over time during training are listed in figure 3.

4.3 LightGCN

The model achieved the best results. Our best performing hyperparameters give a recall value of 0.1039. This better performance is expected since the simplifications of Light GCN also serve to both model performance. The other top performing hyperparamter combos are listed in table 8. The validation curves during training and the loss curves are illustrated in figures 4 and 5.

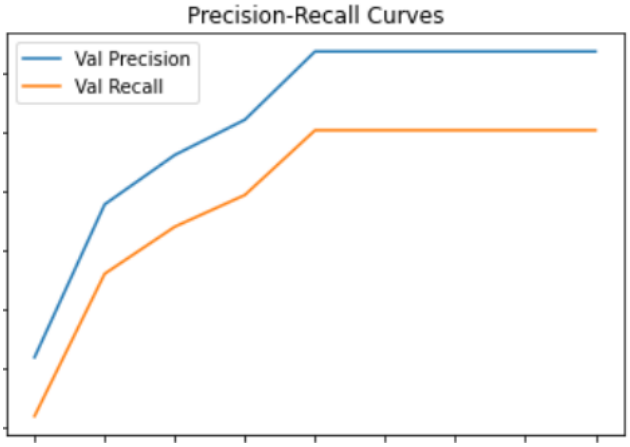


Figure 3: Validation precision and recall curves during training For the best performing NGCF model.

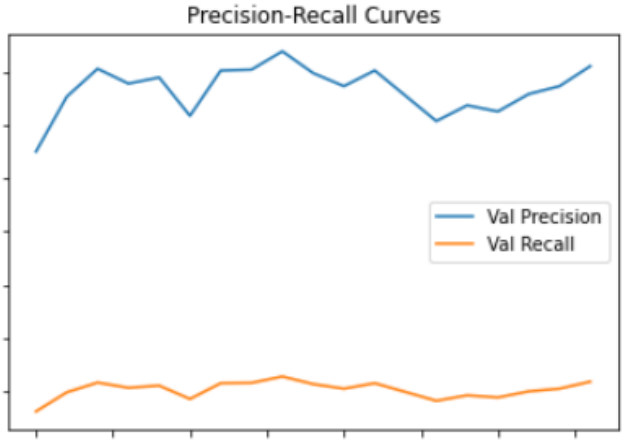


Figure 4: Validation precision and recall curves during training For one of the LightGCN models.

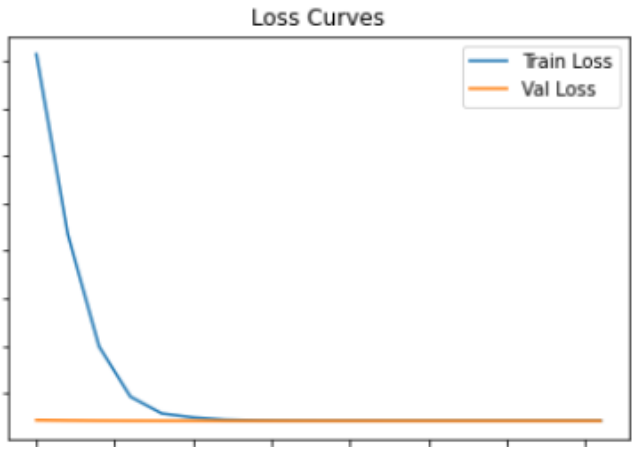


Figure 5: Training and validation loss over time for the Light GCN model.

4.4 Movie Recommendation Application

Once we had trained the best models we developed a simple UI using streamlit where the user could input the IDs of the people one wanted to recommend movies for and also the number of movies one wanted to recommend and the app would then display the results for those users.This application uses the LightGCN model to make predictions.

We built this model for a couple of reasons. First, we wanted to have practice with making a real-life application for a machine learning model. Second, this application served as a bit of a sanity check as it allowed us to manually see what our best performing model is predicting and check if the results make sense for specific users.

5 Conclusions

We found that the LightGCN performed significantly better than the NGCF when we tested the precision@5 and recall@5 for various parameters. This was a win-win since the LightGCN was computationally less expensive and also more accurate.But we did find that LightGCN can overfit so it is necessary to regularize it well.

We found that shallow embedding also performed very poorly on this dataset since it does not directly model the graph structure.

We had a couple of tasks that we wanted to complete for our project but ran out of time in the end. As previously mentioned, we were not able to

get the GC-MC working because we found that it was predicting a rating of 5 for each of the edges. But we are continuing to investigate it to see if it can give better results for this problem. We also would like to publicly host our web application so that it can be easily accessed to view results for this data set. Finally, we would like to gather other datasets to test our models on. This could reveal more information about the strengths and weaknesses of each of them. We leave these tasks for future work.

Overall, we are very happy with how our models have performed. Each of them performs approximately as expected when compared with their respective papers. The limitation of these models are the same as any other recommendation system's limitation - it is incredibly difficult to detect what movie a user wants to see next.

Thank you professor Azad for a great semester!

Acknowledgements

We would like to thank Professor Azad for his work in teaching this course - ENGR E-536. We also thank TAs for this class for helping manage the homework and grading. Without their contribution, we would not have been able to achieve such a great learning experience this semester.

References

[1] Wang, Shoujin, et al. "Graph learning based recommender systems: A review." arXiv preprint arXiv:2105.06339 (2021).

[2] Wang, Xiang, et al. "Neural graph collaborative filtering." Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval. 2019.

[3] He, Xiangnan, et al. "Lightgcn: Simplifying and powering graph convolution network for recommendation." Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval. 2020.

[4] Berg, Rianne van den, Thomas N. Kipf, and Max Welling. "Graph convolutional matrix completion." arXiv preprint arXiv:1706.02263 (2017).

[5] Mikolov, Tomas, et al. "Efficient estimation of word representations in vector space." arXiv preprint arXiv:1301.3781 (2013).

[6] Harper, F. Maxwell, and Joseph A. Konstan. "The movielens datasets: History and context." Acm transactions on interactive intelligent systems (tiis) 5.4 (2015): 1-19.

[7] Azad, Ariful. "Lecture 6: Node Embeddings." ENGR-E536, Spring Semester. 2023.

[8] Azad, Ariful. "Lecture 13: Recommendation Systems." ENGR-E536, Spring Semester. 2023.